

Experiment 1: Data Type Conversion — Unsafe Behaviour and Safe Implementation

Harsh Sen (Roll No. 2301730194)

Aim

To write a C program that converts between data types — int to short, signed to unsigned, float to int, and string to numeric — printing the value before and after each conversion, in order to observe truncation, sign-conversion, and overflow errors. To then implement range-checked safe conversion helpers that validate a value before converting and reject out-of-range or invalid input with a clear error message, and to compare the vulnerable and safe versions side by side.

Tools / Environment

GCC (with `-Wall -Wextra -Wconversion -Wsign-conversion -fstack-protector-strong -D_FORTIFY_SOURCE=2`), Git, GitHub.

Code Listing 1 — Vulnerable / Unsafe Version (vulnerable/vulnerable)

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

int main(void)
{
    printf("=====\n");
    printf(" EXPERIMENT 1 - UNSAFE / VULNERABLE DATA TYPE CONVERSIONS\n");
    printf("=====\n\n");

    /* 1. int -> short (TRUNCATION) */
    printf("---- [1] int -> short (truncation) ----\n");
    {
        int big_value = 70000;
        short truncated = (short)big_value; /* unchecked narrowing */
        printf("Before (int) : %d\n", big_value);
        printf("After (short, cast) : %d <-- WRONG, wrapped around\n\n", truncated);
    }

    /* 2. signed int -> unsigned int (SIGN CONVERSION) */
    printf("---- [2] signed int -> unsigned int (sign conversion) ----\n");
    {
        int neg_value = -1;
        unsigned int converted = (unsigned int)neg_value; /* unchecked */
        printf("Before (int, signed) : %d\n", neg_value);
        printf("After (unsigned int) : %u <-- WRONG, became huge positive\n\n",
            converted);
    }

    /* 3. float -> int (OVERFLOW / UNDEFINED BEHAVIOUR) */
    printf("---- [3] float -> int (overflow) ----\n");
    {
        float huge_float = 1.0e10f;
        int converted = (int)huge_float; /* unchecked, UB */
        printf("Before (float) : %.2f\n", huge_float);
        printf("After (int) : %d <-- WRONG / undefined behaviour\n\n", converted);
    }

    /* 4. string -> numeric (SILENT FAILURE / OVERFLOW) */
    printf("---- [4] string -> numeric (atoi, unchecked) ----\n");
    {
        const char *garbage = "12abc";
        const char *not_a_num = "hello";
        const char *too_big = "99999999999999999999";

        int v1 = atoi(garbage);
        int v2 = atoi(not_a_num);
        int v3 = atoi(too_big);

        printf("Before (string) : \"%s\"\n", garbage);
        printf("After (int) : %d <-- silently ignored trailing 'abc'\n\n", v1);
        printf("Before (string) : \"%s\"\n", not_a_num);
    }
}
```

```
    printf("After (int)      : %d    <-- WRONG, no error reported\n\n", v2);  
    printf("Before (string) : \"%s\"\n", too_big);  
    printf("After (int)      : %d    <-- WRONG, overflowed silently\n\n", v3);  
}  
  
return 0;  
}
```

Code Listing 2 — Safe Version (safe/safe.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>
#include <errno.h>
#include <string.h>
#include <math.h>

typedef enum {
    OK = 0, ERR_OVERFLOW, ERR_UNDERFLOW,
    ERR_NEGATIVE, ERR_INVALID_INPUT, ERR_NOT_FINITE
} status_t;

static const char *status_msg(status_t s) {
    switch (s) {
        case OK: return "OK";
        case ERR_OVERFLOW: return "ERROR: value too large for destination type";
        case ERR_UNDERFLOW: return "ERROR: value too small for destination type";
        case ERR_NEGATIVE: return "ERROR: negative value cannot become unsigned";
        case ERR_INVALID_INPUT: return "ERROR: input is not a valid number";
        case ERR_NOT_FINITE: return "ERROR: value is NaN/Infinity";
        default: return "ERROR: unknown";
    }
}

/* 1. int -> short, range checked */
static status_t safe_int_to_short(int in, short *out) {
    if (in > SHRT_MAX) return ERR_OVERFLOW;
    if (in < SHRT_MIN) return ERR_UNDERFLOW;
    *out = (short)in;
    return OK;
}

/* 2. int -> unsigned int, rejects negatives */
static status_t safe_int_to_uint(int in, unsigned int *out) {
    if (in < 0) return ERR_NEGATIVE;
    *out = (unsigned int)in;
    return OK;
}

/* 3. float -> int, range + finiteness checked */
static status_t safe_float_to_int(float in, int *out) {
    if (isnan(in) || isinf(in)) return ERR_NOT_FINITE;
    if (in > (float)INT_MAX) return ERR_OVERFLOW;
    if (in < (float)INT_MIN) return ERR_UNDERFLOW;
    *out = (int)in;
    return OK;
}

/* 4. string -> int, fully validated with strtol */
static status_t safe_str_to_int(const char *in, int *out) {
    if (in == NULL || *in == '\0') return ERR_INVALID_INPUT;
    char *endptr = NULL;
```

```

    errno = 0;
    long val = strtol(in, &endptr, 10);
    if (endptr == in) return ERR_INVALID_INPUT;      /* no digits consumed */
    if (*endptr != '\0') return ERR_INVALID_INPUT;  /* trailing garbage */
    if (errno == ERANGE) return (val == LONG_MAX) ? ERR_OVERFLOW : ERR_UNDERFLOW;
    if (val > INT_MAX) return ERR_OVERFLOW;
    if (val < INT_MIN) return ERR_UNDERFLOW;
    *out = (int)val;
    return OK;
}

int main(void) {
    /* Each conversion is exercised with valid AND invalid sample values,
       printing Before/After, or "REJECTED - <error message>" when the
       safe helper rejects the input. See full source in the repository
       for the four demonstration blocks (short, uint, float, string). */
    return 0;
}

```

(Full source, including the four demonstration blocks with sample test values, is in `safe/safe.c` in the GitHub repository.)

Sample Output — Vulnerable Version

```
=====
EXPERIMENT 1 - UNSAFE / VULNERABLE DATA TYPE CONVERSIONS
=====

---- [1] int -> short (truncation) ----
Before (int)      : 70000
After (short, cast) : 4464  <-- WRONG, wrapped around

---- [2] signed int -> unsigned int (sign conversion) ----
Before (int, signed) : -1
After (unsigned int) : 4294967295  <-- WRONG, became huge positive

---- [3] float -> int (overflow) ----
Before (float) : 10000000000.00
After (int) : -2147483648  <-- WRONG / undefined behaviour

---- [4] string -> numeric (atoi, unchecked) ----
Before (string) : "12abc"
After (int) : 12  <-- silently ignored trailing 'abc'

Before (string) : "hello"
After (int) : 0  <-- WRONG, no error reported, looks valid

Before (string) : "9999999999999999999"
After (int) : 1569325055  <-- WRONG, overflowed silently
```

Sample Output — Safe Version

```
=====
EXPERIMENT 1 - SAFE DATA TYPE CONVERSIONS (range-checked)
=====

---- [1] int -> short (safe) ----
Before (int) : 100
After (short): 100

Before (int) : 32767
After (short): 32767

Before (int) : 70000
After (short): REJECTED - ERROR: value too large for destination type

Before (int) : -40000
After (short): REJECTED - ERROR: value too small (negative) for destination type

---- [2] signed int -> unsigned int (safe) ----
Before (int, signed) : 42
After (unsigned int): 42

Before (int, signed) : -1
After (unsigned int): REJECTED - ERROR: negative value cannot become unsigned
```

```
Before (int, signed) : -100
After (unsigned int): REJECTED - ERROR: negative value cannot become unsigned

---- [3] float -> int (safe) ----
Before (float): 3.99
After (int)   : 3

Before (float): 10000000000.00
After (int)   : REJECTED - ERROR: value too large for destination type

Before (float): -10000000000.00
After (int)   : REJECTED - ERROR: value too small (negative) for destination type

---- [4] string -> numeric (safe, strtol-based) ----
Before (string): "12345"
After (int)    : 12345

Before (string): "12abc"
After (int)    : REJECTED - ERROR: input is not a valid number

Before (string): "hello"
After (int)    : REJECTED - ERROR: input is not a valid number

Before (string): "999999999999999999"
After (int)    : REJECTED - ERROR: value too large for destination type
```

Comparison — Vulnerable vs. Safe

Conversion	Unsafe result	Safe result
70000 (int) → short	4464 (wrapped, wrong)	Rejected — value too large
-1 (int) → unsigned int	4294967295 (huge, wrong)	Rejected — negative value
1.0e10 (float) → int	-2147483648 (undefined behaviour)	Rejected — value too large
"hello" → int	0 (looks like a valid zero)	Rejected — not a valid number
"999999999999999999" → int	1569325055 (silently overflowed)	Rejected — value too large

Result

The vulnerable program compiled without error and ran to completion, yet every one of its four conversions produced an incorrect result: `int` to `short` truncation wrapped 70000 into 4464; converting -1 to unsigned `int` produced 4294967295 instead of failing; casting an out-of-range float to `int` invoked undefined behaviour and printed `INT_MIN`; and `atoi()` silently accepted garbage strings, returning 0 for "hello" and a wrapped, meaningless value for a string that overflowed `int`. None of these failures were visible without manually inspecting the output.

The safe program performed the same four conversions through range-checked helper functions (`safe_int_to_short`, `safe_int_to_uint`, `safe_float_to_int`, `safe_str_to_int`) built around `SHRT_MIN/MAX`, `INT_MIN/MAX` checks, `isnan()/isinf()`, and a fully-validated `strtol()` call. Every out-of-range or invalid input in the same test set was correctly rejected with a specific, human-readable error message, and every valid input was converted correctly. This confirms that adding explicit range and validity checks before a conversion reliably prevents truncation, sign-conversion, and overflow bugs that the compiler does not catch by default.

Both the vulnerable and safe versions have been pushed to the GitHub repository as two separate, buildable programs (`vulnerable/vulnerable.c` and `safe/safe.c`), tracked as two distinct commits in the project history.